

Akash Goda
ECON 580
13 March 2026
Assignment 5

Introduction

A growing body of empirical literature estimates large gross productivity gains from artificial intelligence in knowledge work. Peng et al. (2023) find that developers using GitHub Copilot, an AI code-completion tool, finish tasks 55.8% faster in a randomized controlled trial (RCT). Yeverechyahu et al. (2024) exploit the same natural experiment proposed below and find a 37–54% increase in commit output on GitHub following Copilot’s general release. These are task-level estimates which feed directly into broader macroeconomic forecasts. Acemoglu (2024) uses the gross productivity gains of this kind as inputs for a TFP (total factor productivity) growth model, and the 0.66% ten-year TFP growth is widely cited as a result. A critical limitation which is shared by the majority of these studies is that they measure *quantity* of output, rather than the *quality*. Peng et al. explicitly state that they “do not examine code quality.” Yeverechyahu et al. count commits without assessing whether those commits survive in the codebase. What this poses is that if AI-assisted output requires highly-substantive downstream rework, then the gross gains described in mainstream literature are systematically overstated and that productivity estimates embedded in current TFP forecasts are biased upward. In addition to this, the descriptive evidence at first glance suggests that this concern is not hypothetical. Software-as-a-service provider GitClear (2024, 2025) reports that code churn, defined as code written and then subsequently rewritten shortly after, roughly doubled in the post-Copilot period. After conducting an RCT, Becker (2025) found experienced developers using AI tools were 19% *slower* on real-world tasks despite believing they were 20% faster. However, neither study provides a causal estimate which is suitable for adjusting productivity models.

Research Question

Does AI-assisted production increase output quantity while degrading output quality, and if so, how much do gross task-level productivity estimates overstate net gains? My contribution is threefold. First, a validated natural experiment is applied to measure quality instead of quantity. Second, a code-quality penalty metric that can be plugged back into Acemoglu's TFP framework to calculate a true net-productivity gain is provided. Third, there is testing to see if this quality penalty varies across different software subindustries.

Literature Review - Causal Estimates of AI Coding Productivity

Three studies provide causal estimates regarding how AI code-generation tools affect developer productivity, and they all measure quantity. Firstly, Peng et al. (2023) conducted an RCT at Microsoft in which 95 developers were assigned to either use GitHub Copilot or a manual control condition. Developers with Copilot completed the standardized HTTP server task 55.8% faster. The internal validity of this experiment is strong, however the external validity is narrow as the task was a singular exercise. Participants were drawn from one company and the

authors of the study explicitly indicate code quality was not considered. Yeverechyahu, Mayya, and Oestreicher-Singer (2024) take an observational approach using the same natural experiment I exploited below. When GitHub Copilot launched in June 2022, it supported some programming languages well (Python, JavaScript, TypeScript) and others poorly (R, Haskell), determined by training data availability rather than user demand. Using this differential exposure in a difference-in-differences framework, they find a 37–54% increase in commits for treated languages relative to control languages. This result is critical for my paper because it validates that the language split generates a real first stage in AI exposure. However, the authors solely measure commit volume and not whether the commits persist or require rework. Becker et al. (2025) conduct an RCT through the research organization METR with 16 experienced open-source developers working on their own real repositories. They find that AI tools made developers 19% slower on average, despite participants self-reporting that they felt 20% faster. This is the only existing study that hints at a quality channel: the slowdown likely reflects time spent reviewing and debugging AI-generated suggestions. However, the sample is very small, restricted to senior developers, and the study does not directly measure code quality or churn.

Literature Review - Descriptive Evidence & Theoretical Frameworks

GitClear (2024), a software analytics firm, published an industry report analyzing 211 million lines of code across client repositories. They report code churn, defined as code written and then rewritten shortly after, roughly doubled between 2021 and 2023. This coincides with widespread adoption of Copilot. While suggestive, the evidence is purely correlational at surface level, as there is no control group and no identification strategy. The pandemic is also an attribute that has not been considered. Acemoglu (2024, NBER Working Paper No. 32487) provides the theoretical framework for the macroeconomic portion of the project. He constructs a TFP growth model using gross task-level cost savings from studies like Peng et al. as inputs and arrives at an upper bound of 0.66% TFP growth over ten years. This estimate has been widely cited in policy discussions. Crucially, the model takes gross productivity gains at face value. If the task-level inputs are overstated because they ignore downstream rework costs, the TFP ceiling is even lower than 0.66%.

Data & Sample Construction

Data is derived from GitHub Archive via Google BigQuery, which is a free public dataset which records every event on every public GitHub repository since 2011. The dataset is queryable via SQL and includes commit-level event data. The unit of observation is the repo-month. Several sample restrictions to isolate repositories where code quality plausibly matters. The first restriction is repositories with a minimum of 10 stars and 2 unique contributors over the sample period. This is intended to filter out personal projects, homework assignments, inactive forks, as well as auto-generated repositories. The second protocol is to drop repositories with under 5 commit-months of activity in the pre-period of January 2020-May 2022. This is to ensure there is sufficient data for estimating fixed effects. The primary motive is retaining repositories that represent genuine collaboration in software development. Repositories will be classified into treatment and control groups by primary language, and for repositories with

multiple languages, this is defined by the language with the most code by bytes as reported. The treatment languages are Python, JavaScript, and TypeScript while the control languages are R and Haskell. If further robustness is required, a threshold of 70% of code in the primary language shall be added.

Identification Strategy & Estimation

$$ChurnRate_{it} = \alpha_i + \gamma_t + \beta(Treated_i \times Post_t) + \delta_1 \ln(Commits_{it}) + \delta_2 \ln(Contributors_{it}) + \varepsilon_{it}$$

Using difference-in-differences with two-way fixed effects, I estimate the above specification, where ChurnRate is the proportion of modified files in repository i in month t that are re-modified within 72 hours. Using a rate rather than a count controls for the mechanical relationship between activity level and churn. α are repository fixed effects, which absorb all time-invariant repository characteristics including primary language, project type, baseline code quality, and developer skill composition. These also absorb the *Treated* main effect. γ are year-month fixed effects, which absorb common shocks affecting all repositories such as macroeconomic conditions, industry-wide trends, and seasonal patterns. These also absorb the *Post* main effect. The coefficient of interest is β , which captures the differential change in churn rate for Copilot-exposed repositories relative to control repositories after June 2022, net of repo-specific baselines and common time trends. A positive β implies that AI-assisted code requires more rework. Also included are $\ln(Commits)$ and $\ln(Contributors)$ as time-varying controls for repository activity and team size, logged to address right-skew. These account for the possibility that Copilot may increase commit volume in treated repos, which could mechanically affect churn if not controlled for. Standard errors are clustered at the repository level to account for serial correlation within repos over time.

Identification Threats & Robustness

The primary threat to identification is contamination from the launch of ChatGPT on November 30, 2022 which occurred 5 months after Copilot. ChatGPT is language-agnostic unlike Copilot, which means that developers using R or Haskell can paste code into the chat interface and receive AI-generated suggestions. If this pushes the control group churn upward, it attenuates β toward zero, making my estimate a conservative lower bound. The first way this is addressed is that the primary specification utilizes only the clean June-November 2022 window before ChatGPT existed. The second is through utilizing placebo tests that shift the treatment date to arbitrary months where no AI tool launched such as January 2021 and January 2022. Finding effects at fake dates would indicate problems with the study design. Lastly, event study calculations can be explored in later iterations of the project drafts. A second concern is the choice of churn window. One peer reviewer noted that a strict 72-hour window may miss buggy code left over a weekend, while another noted that rapid rewrites could reflect deliberate experimentation rather than defects. I address the first concern by running robustness checks with 48-hour, 96-hour, and 7-day windows. I address the second by noting that the $\ln(Commits_{it})$ control absorbs activity-level variation: repositories where developers are rapidly iterating will have high commit counts, and the churn rate conditional on commit volume isolates rework from experimentation.

References

Acemoglu, D. (2024). The simple macroeconomics of AI (NBER Working Paper No. 32487). National Bureau of Economic Research. <https://doi.org/10.3386/w32487>

GitClear. (2024). Coding on Copilot: 2023 data suggests downward pressure on code quality. GitClear Research.

Becker, J., Rush, N., Barnes, E., & Rein, D. (2025). Measuring the impact of early-2025 AI on experienced open-source developer productivity. arXiv. <https://arxiv.org/abs/2507.09089>

Peng, S., Kalliamvakou, E., Cihon, P., & Demirer, M. (2023). The impact of AI on developer productivity: Evidence from GitHub Copilot. arXiv. <https://arxiv.org/abs/2302.06590>

Yeverechyahu, D., Mayya, R., & Oestreicher-Singer, G. (2024). The impact of large language models on open-source innovation: Evidence from GitHub Copilot. arXiv. <https://arxiv.org/abs/2409.08379>